

C Program for Depth First Search or DFS for a Graph

Last Updated : 09 Nov, 2023



Depth First Traversal (or DFS) for a graph is similar to [Depth First Traversal of a tree](#). The only catch here is, that, unlike trees, graphs may contain cycles (a node may be visited twice). To avoid processing a node more than once, use a boolean visited array. A graph can have more than one DFS traversal.

Example:

Input: $n = 4, e = 6$
 $0 \rightarrow 1, 0 \rightarrow 2, 1 \rightarrow 2, 2 \rightarrow 0, 2 \rightarrow 3, 3 \rightarrow 3$
Output: DFS from vertex 1 : 1 2 0 3

Input: $n = 4, e = 6$
 $2 \rightarrow 0, 0 \rightarrow 2, 1 \rightarrow 2, 0 \rightarrow 1, 3 \rightarrow 3, 1 \rightarrow 3$
Output: DFS from vertex 2 : 2 0 1 3

How does DFS work?

Depth-first search is an algorithm for traversing or searching tree or graph data structures. The algorithm starts at the root node (selecting some arbitrary node as the root node in the case of a graph) and explores as far as possible along each branch before backtracking.

Below is the implementation of the above approach:

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

// Structure for a node in the adjacency list
struct Node {
    int data;
    struct Node* next;
};
```

```

// Structure for the adjacency list
struct List {
    struct Node* head;
};

// Structure for the graph
struct Graph {
    int vertices;
    struct List* array;
};

// Function to create a new node
struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct
Node));
    newNode->data = data;
    newNode->next = NULL;
    return newNode;
}

// Function to create a graph with a given number of vertices
struct Graph* createGraph(int vertices) {
    struct Graph* graph = (struct Graph*)malloc(sizeof(struct
Graph));
    graph->vertices = vertices;
    graph->array = (struct List*)malloc(vertices * sizeof(struct
List));

    for (int i = 0; i < vertices; i++) {
        graph->array[i].head = NULL;
    }

    return graph;
}

// Function to add an edge to the graph
void addEdge(struct Graph* graph, int src, int dest) {
    struct Node* newNode = createNode(dest);
    newNode->next = graph->array[src].head;
    graph->array[src].head = newNode;
}

```

```

// Uncomment the following code to make the graph undirected
/*
newNode = createNode(src);
newNode->next = graph->array[dest].head;
graph->array[dest].head = newNode;
*/
}

// Function to perform Depth First Search (DFS) from a given
vertex
void DFS(struct Graph* graph, int vertex, bool visited[]) {
    visited[vertex] = true;
    printf("%d ", vertex);

    struct Node* currentNode = graph->array[vertex].head;
    while (currentNode) {
        int adjacentVertex = currentNode->data;
        if (!visited[adjacentVertex]) {
            DFS(graph, adjacentVertex, visited);
        }
        currentNode = currentNode->next;
    }
}

// Function to perform DFS traversal from a given vertex in a
specified order
void DFSTraversal(struct Graph* graph, int* order, int orderSize)
{
    bool* visited = (bool*)malloc(graph->vertices * sizeof(bool));
    for (int i = 0; i < graph->vertices; i++) {
        visited[i] = false;
    }

    for (int i = 0; i < orderSize; i++) {
        if (!visited[order[i]]) {
            DFS(graph, order[i], visited);
        }
    }

    free(visited);
}

```

```

int main() {
    int vertices = 4;
    struct Graph* graph = createGraph(vertices);

    addEdge(graph, 2, 0);
    addEdge(graph, 0, 2);
    addEdge(graph, 1, 2);
    addEdge(graph, 0, 1);
    addEdge(graph, 3, 3);
    addEdge(graph, 1, 3);

    int order[] = {2, 0, 1, 3};
    int orderSize = sizeof(order) / sizeof(order[0]);

    printf("Following is Depth First Traversal (starting from
vertex 2):\n");
    DFSTraversal(graph, order, orderSize);

    return 0;
}

```

Output

```

Following is Depth First Traversal (starting from vertex 2):
2 0 1 3

```

Time Complexity: $O(V+E)$ where V is the number of vertices in the graph and E is the number of edges

Auxiliary Space: $O(V+E)$

Please refer complete article on [Depth First Search or DFS for a Graph](#) for more details!

 Comment

More info ▼

Advertise with us

Next Article >

Depth First Search or DFS for a Graph -
Python

Similar Reads

Depth First Search or DFS for a Graph

In Depth First Search (or DFS) for a graph, we traverse all adjacent vertices one by one. When we traverse an adjacent vertex, we completely finish the traversal of all vertices reachable through that...

🕒 13 min read

DFS in different language



C Program for Depth First Search or DFS for a Graph

Depth First Traversal (or DFS) for a graph is similar to Depth First Traversal of a tree. The only catch here is, that, unlike trees, graphs may contain cycles (a node may be visited twice). To avoid processing a...

🕒 4 min read

Depth First Search or DFS for a Graph - Python

Depth First Traversal (or DFS) for a graph is similar to Depth First Traversal of a tree. The only catch here is, that, unlike trees, graphs may contain cycles (a node may be visited twice). To avoid processing a...

🕒 4 min read

Java Program for Depth First Search or DFS for a Graph

Depth First Traversal (or DFS) for a graph is similar to Depth First Traversal of a tree. Prerequisite: Graph knowledge is important to understand the concept of DFS. What is DFS? DFS or Depth First Traversal i...

🕒 3 min read

Iterative Depth First Traversal of Graph

Given a directed Graph, the task is to perform Depth First Search of the given graph. Note: Start DFS from node 0, and traverse the nodes in the same order as adjacency list. Note : There can be multiple DFS...

🕒 10 min read

Applications, Advantages and Disadvantages of Depth First Search (DFS)

Depth First Search is a widely used algorithm for traversing a graph. Here we have discussed some applications, advantages, and disadvantages of the algorithm. Applications of Depth First Search: 1....

🕒 4 min read

Difference between BFS and DFS

Breadth-First Search (BFS) and Depth-First Search (DFS) are two fundamental algorithms used for traversing or searching graphs and trees. This article covers the basic difference between Breadth-First...

🕒 2 min read

Depth First Search or DFS for disconnected Graph

Given a Disconnected Graph, the task is to implement DFS or Depth First Search Algorithm for this Disconnected Graph. Example: Input: Disconnected Graph Output: 0 1 2 3 Algorithm for DFS on...

🕒 7 min read

Printing pre and post visited times in DFS of a graph

Depth First Search (DFS) marks all the vertices of a graph as visited. So for making DFS useful, some additional information can also be stored. For instance, the order in which the vertices are visited while...

🕒 8 min read

Tree, Back, Edge and Cross Edges in DFS of Graph

Given a directed graph, the task is to identify tree, forward, back and cross edges present in the graph. Note: There can be multiple answers. Example: Input: Graph Output: Tree Edges: 1->2, 2->4, 4->6, ...

🕒 9 min read

Transitive Closure of a Graph using DFS

Given a directed graph, find out if a vertex v is reachable from another vertex u for all vertex pairs (u, v) in the given graph. Here reachable means that there is a path from vertex u to v . The reach-ability matrix is...

🕒 8 min read

Variations of DFS implementations

